Representation of Data

Binary · Octal · Hexadecimal · BCD · EBCDIC · ASCII

Number System Conversions & Arithmetic Operations

Computer Science — Data Representation

Master Number Reference Table (0 – 15)

Each row shows a value 0–15 in **Decimal, 8-bit Binary, Octal, Hexadecimal, BCD** and its **Control character name**. These 16 values are the foundation of all number system work.

Dec	Binary (8-bit)	Octal	Hex	BCD	Control Name
0	0000 0000	0	0	0000	NUL
1	0000 0001	1	1	0001	SOH
2	0000 0010	2	2	0010	STX
3	0000 0011	3	3	0011	ETX
4	0000 0100	4	4	0100	EOT
5	0000 0101	5	5	0101	ENQ
6	0000 0110	6	6	0110	ACK
7	0000 0111	7	7	0111	BEL
8	0000 1000	10	8	1000	BS
9	0000 1001	11	9	1001	HT
10	0000 1010	12	A	0001 0000	LF
11	0000 1011	13	B	0001 0001	VT
12	0000 1100	14	C	0001 0010	FF
13	0000 1101	15	D	0001 0011	CR
14	0000 1110	16	E	0001 0100	SO
15	0000 1111	17	F	0001 0101	SI

1. Binary Number System (Base-2)

The binary system uses only **0** and **1**. Every digital circuit represents two states — OFF (0) and ON (1). All computer data is ultimately stored and processed in binary.

Terminology

Term	Meaning
Bit	Single binary digit (0 or 1) — smallest data unit
Nibble	Group of 4 bits e.g. 1010
Byte	Group of 8 bits e.g. 1100 1010
MSB	Most Significant Bit — leftmost, highest value
LSB	Least Significant Bit — rightmost, lowest value

Positional Value (powers of 2)

Bit Position	7	6	5	4	3	2	1	0
Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	128	64	32	16	8	4	2	1

Example: $1011_2 = 1\times8 + 0\times4 + 1\times2 + 1\times1 = \mathbf{11_{10}}$

2. Octal Number System (Base-8)

The octal system uses digits **0 through 7**. Each octal digit maps exactly to **3 binary bits**, making binary-octal conversion trivial.

Octal Digit to Binary Mapping

Octal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

Positional Example: $(253)_8 = 2\times8^2 + 5\times8^1 + 3\times8^0 = 128 + 40 + 3 = \mathbf{171_{10}}$

3. Hexadecimal Number System (Base-16)

The hexadecimal system uses 16 symbols: **0–9** and **A–F** (A=10, B=11, C=12, D=13, E=14, F=15). Each hex digit = **4 binary bits**. Used extensively in memory addresses, colour codes and assembly language.

Hex Digit Reference Table

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
4-bit Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Example: $(2AF)_{16} = 2\times16^2 + 10\times16^1 + 15\times16^0 = 512 + 160 + 15 = \mathbf{687_{10}}$

4. Number System Conversions

4.1 Decimal to Binary — Repeated Division by 2

Divide by 2 repeatedly; record each remainder. Read remainders **bottom to top** for the binary result.

Step	Division	Quotient	Remainder
1	45 ÷ 2	22	1 ← LSB
2	22 ÷ 2	11	0
3	11 ÷ 2	5	1
4	5 ÷ 2	2	1
5	2 ÷ 2	1	0
6	1 ÷ 2	0	1 ← MSB

Result: $45_{10} = \mathbf{101101_2}$

4.2 Binary to Decimal — Sum of Positional Values

Multiply each bit by its positional power of 2, then sum all values.

$101101_2 = 1\times32 + 0\times16 + 1\times8 + 1\times4 + 0\times2 + 1\times1 = 32+8+4+1 = \mathbf{45_{10}}$

4.3 Decimal to Octal — Repeated Division by 8

Step	Division	Quotient	Remainder
1	171 ÷ 8	21	3 ← LSD
2	21 ÷ 8	2	5
3	2 ÷ 8	0	2 ← MSD

Result: 171_{10} = **$(253)_8$**

4.4 Octal ↔ Binary

Octal to Binary: Replace each octal digit with its 3-bit equivalent.

$(253)_8$ → 2 = 010, 5 = 101, 3 = 011 → **$010\ 101\ 011_2$**

Binary to Octal: Group bits into sets of 3 from right; convert each.

4.5 Decimal to Hexadecimal — Repeated Division by 16

Step	Division	Quotient	Remainder
1	687 ÷ 16	42	15 → F ← LSD
2	42 ÷ 16	2	10 → A
3	2 ÷ 16	0	2 ← MSD

Result: 687_{10} = **$(2AF)_{16}$**

4.6 Hex ↔ Binary

Hex to Binary: Replace each hex digit with its 4-bit equivalent.

$(2AF)_{16}$ → 2=0010, A=1010, F=1111 → **$0010\ 1010\ 1111_2$**

Binary to Hex: Group bits into sets of 4 from right; convert each.

4.7 Octal ↔ Hexadecimal (via Binary)

Convert Octal → Binary (3-bit groups) → regroup into 4-bit groups → Hexadecimal.

Example: $(253)_8$ = $010\ 101\ 011_2$ = $0\ 1010\ 1011_2$ = **$(0AB)_{16}$**

5. BCD — Binary Coded Decimal

BCD encodes each individual **decimal digit (0–9)** as a separate 4-bit binary group. It is **not** the same as converting the complete number to binary.

BCD Encoding Table

Decimal Digit	BCD Code	Decimal Digit	BCD Code
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Worked Examples

Number	BCD Encoding	Result
396	3→0011 | 9→1001 | 6→0110	0011 1001 0110
57	5→0101 | 7→0111	0101 0111
BCD 0101 0111	0101=5 | 0111=7	57_{10}

BCD vs Pure Binary Comparison

Feature	BCD	Pure Binary
Encoding of digit 9	1001	1001
Encoding of 99	1001 1001 (8 bits)	110 0011 (7 bits)
Storage efficiency	Less compact	More compact
Primary use	Calculators, 7-seg displays	All computer arithmetic

Important: BCD codes 1010 through 1111 are **invalid** — no decimal digit maps to 10–15.

6. EBCDIC — Extended Binary Coded Decimal Interchange Code

EBCDIC is an 8-bit character encoding developed by **IBM in 1964** for use on System/360 mainframes. It supports **256 characters**.

Key Characteristics

- **8 bits** per character — 256 possible values.
- Letters are **non-contiguous**: A–I (C1–C9), J–R (D1–D9), S–Z (E2–E9) occupy three separate ranges.
- Digits 0–9 encoded as **F0–F9** (hex).
- Still used on IBM mainframes and AS/400 systems today.

EBCDIC Character Values

Character	Hex	Binary (8-bit)	Notes
A	C1	1100 0001	Start of uppercase A–I range
B	C2	1100 0010	
Z	E9	1110 1001	End of uppercase S–Z range
a	81	1000 0001	Start of lowercase a–i range
z	A9	1010 1001	End of lowercase s–z range
0	F0	1111 0000	Start of digit range
9	F9	1111 1001	End of digit range
Space	40	0100 0000	Blank character

EBCDIC vs ASCII

Feature	EBCDIC	ASCII
Bits per character	8	7 (extended: 8)
Total characters	256	128 (extended: 256)
Developed by	IBM (1964)	ANSI / ISO (1963)
Letter arrangement	Non-contiguous	Contiguous (A=65 to Z=90)
Digit range	F0–F9 (hex)	30–39 (hex)
Primary platform	IBM mainframes	Universal / PCs / Internet

7. ASCII — American Standard Code for Information Interchange

ASCII is a 7-bit character encoding standard covering **128 characters** (codes 0–127): control codes, digits, punctuation, and both cases of the English alphabet. Extended ASCII uses 8 bits (256 chars).

ASCII Range Structure

Range (Dec)	Range (Hex)	Character Group	Examples
0 – 31	00 – 1F	Control characters	NUL, HT, LF, CR, ESC
32	20	Space	(blank)
33 – 47	21 – 2F	Punctuation / symbols	! " # $ % & ' ( )
48 – 57	30 – 39	Digits	0 1 2 3 4 5 6 7 8 9
58 – 64	3A – 40	Punctuation	: ; < = > ? @
65 – 90	41 – 5A	Uppercase letters	A B C … X Y Z
91 – 96	5B – 60	Symbols	[\ ] ^ _ `
97 – 122	61 – 7A	Lowercase letters	a b c … x y z
123 – 127	7B – 7F	Symbols & DEL	{ | } ~ DEL

Key ASCII Values — Quick Reference

Char	Dec	Hex	8-bit Binary	Char	Dec	Hex	8-bit Binary
A	65	41	0100 0001	a	97	61	0110 0001
B	66	42	0100 0010	b	98	62	0110 0010
M	77	4D	0100 1101	m	109	6D	0110 1101
Z	90	5A	0101 1010	z	122	7A	0111 1010
0	48	30	0011 0000	9	57	39	0011 1001
Space	32	20	0010 0000	DEL	127	7F	0111 1111

Key rules: 'A' = 65, 'a' = 97. Difference between any uppercase and its lowercase = **32**. Digit '0' = 48, so digit char 'n' = 48 + n.

8. Arithmetic in Octal (Base-8)

All arithmetic follows normal rules but uses base 8. Digits are **0–7 only**. Carries and borrows happen at multiples of 8.

8.1 Addition — when column sum ≥ 8: subtract 8, write remainder, carry 1

Example	Column-by-Column Steps	Result
$(375)_8 + (462)_8$	Units: 5+2 = 7 → write 7, carry 0 8s: 7+6 = 13 → 13–8 = 5, carry 1 64s: 3+4+1 = 8 → 8–8 = 0, carry 1	$(1057)_8$
$(27)_8 + (53)_8$	Units: 7+3 = 10 → 10–8 = 2, carry 1 8s: 2+5+1 = 8 → 8–8 = 0, carry 1	$(102)_8$

8.2 Subtraction — when top digit < bottom: borrow 8 from next column

Example	Column-by-Column Steps	Result
$(732)_8 - (465)_8$	Units: 2 < 5 → borrow 8 → (2+8)–5 = 5 8s: (3–1) = 2 < 6 → borrow 8 → (2+8)–6 = 4 64s: 7–1(borrow)–4 = 2	$(245)_8$
$(54)_8 - (27)_8$	Units: 4 < 7 → borrow 8 → (4+8)–7 = 5 8s: 5–1(borrow)–2 = 2	$(25)_8$

8.3 Multiplication — convert each product to base 8

Example	Steps	Result
$(27)_8 \times (5)_8$	$7\times5 = 35_{10} \rightarrow 35 \div 8 = 4$ rem 3 → write 3, carry 4 $2\times5 = 10_{10} + 4(\text{carry}) = 14_{10} \rightarrow 14 \div 8 = 1$ rem 6 → write 6, carry 1	$(163)_8$
$(13)_8 \times (12)_8$	$(13)_8 \times 2 = 26_{10} = (32)_8$ $(13)_8 \times 1$ (shifted left) $= (130)_8$ $(32)_8 + (130)_8 = (162)_8$	$(162)_8$

8.4 Division — long division; remainder must be less than divisor

Example	Steps	Result
$(162)_8 \div (13)_8$	Convert: $13_8 = 11_{10}$, $16_8 = 14_{10}$ $14 \div 11 = 1$ (quotient digit 1); $1 \times 13_8 = 13_8$; $16_8 - 13_8 = 3_8$ Bring down 2 → $32_8 = 26_{10}$; $26 \div 11 = 2$ (quotient digit 2) $2 \times 13_8 = 26_8$; $32_8 - 26_8 = 4_8$ (remainder)	Q: $(12)_8$ R: $(4)_8$

9. Arithmetic in Hexadecimal (Base-16)

Digits 0–9 and A–F. When a column sum exceeds **15 (F)**: subtract 16, write remainder, carry 1. When borrowing: borrow **16**. Remember: A=10, B=11, C=12, D=13, E=14, F=15.

9.1 Addition

Example	Column-by-Column Steps	Result
$(3A7)_{16} + (2BF)_{16}$	Units: $7+F = 7+15 = 22_{10} \rightarrow 22-16=6$, carry 1 16s: $A+B+1 = 10+11+1 = 22_{10} \rightarrow 22-16=6$, carry 1 256s: $3+2+1 = 6$	$(666)_{16}$
$(9F)_{16} + (A3)_{16}$	Units: $F+3 = 15+3 = 18_{10} \rightarrow 18-16=2$, carry 1 16s: $9+A+1 = 9+10+1 = 20_{10} \rightarrow 20-16=4$, carry 1 → E	$(142)_{16}$

9.2 Subtraction

Example	Column-by-Column Steps	Result
$(7A3)_{16} - (4BF)_{16}$	Units: $3 < F(15)$ → borrow 16 → $(3+16)-15=4$ 16s: $A(10)-1=9 < B(11)$ → borrow 16 → $(9+16)-11=14=E$ 256s: $7-1(\text{borrow})-4=2$	$(2E4)_{16}$
$(C0)_{16} - (3F)_{16}$	Units: $0 < F(15)$ → borrow 16 → $16-15=1$ 16s: $C(12)-1(\text{borrow})=B(11)$; $B-3=8$	$(81)_{16}$

9.3 Multiplication

Example	Steps	Result
$(2A)_{16} \times (3)_{16}$	$A(10) \times 3 = 30_{10} \rightarrow 30-16=14=E$, carry 1 $2 \times 3 = 6 + 1(\text{carry}) = 7$	$(7E)_{16}$
$(1F)_{16} \times (2)_{16}$	$F(15) \times 2 = 30_{10} \rightarrow 30-16=14=E$, carry 1 $1 \times 2 = 2 + 1(\text{carry}) = 3$	$(3E)_{16}$

9.4 Division

Example	Steps	Result
$(7E)_{16} \div (3)_{16}$	$7 \div 3 = 2$ remainder 1 Bring down E: $1E_{16} = 30_{10}$; $30 \div 3 = 10 = A$, rem 0	Q: $(2A)_{16}$ R: 0
$(3E)_{16} \div (2)_{16}$	$3E_{16} = 62_{10}$; $62 \div 2 = 31 = 1F_{16}$	Q: $(1F)_{16}$ R: 0

10. Quick Summary — Number Systems

Feature	Binary	Octal	Hexadecimal	BCD	ASCII	EBCDIC
Base	2	8	16	10 (coded)	—	—
Valid digits	0, 1	0 – 7	0–9, A–F	0 – 9	—	—
Bits per digit	1	3	4	4 / decimal digit	7	8
Carry / borrow at	2	8	16	10	—	—
Primary use	CPU / logic	Legacy systems	Memory / assembly	Displays / BCD arith.	Text on PCs	IBM mainframes

Key Rules to Remember

- **Binary** ↔ **Octal:** group binary bits in sets of **3**.
- **Binary** ↔ **Hex:** group binary bits in sets of **4**.
- **Octal** ↔ **Hex:** always convert via binary as the intermediate.
- **BCD:** codes 1010–1111 are invalid — only 0000–1001 are used.
- **ASCII:** 'A'=65, 'a'=97; uppercase → lowercase by adding **32**; digit '0'=48.
- **EBCDIC:** 'A'=0xC1; digits 0–9 start at 0xF0.